

1.1

All software engineering project handle these basic tasks:

- Gathering requirements
- High-level and low-level design
- Actual development
- Testing
- Deployment
- Maintenance
- Wrap-up

1.2

- Gathering requirements

Working with the customer to turn their desires and ideas into workable, verifiable software objectives that can be clearly pursued and checked off.

- High-level design

Figuring out the surface-level and foundational details of the software, like project architectures, any user interfaces, databases, etc.

- Low-level design

Figuring out more of the specifics of the software, involving details like interface interactions and subcomponents.

- Actual development

Working from the requirements, high- and low-level designs to actually create the software through programming, and preliminary bug fixing.

- Testing

Comprehensive verification that all major possible failures of the software have been addressed, through thorough user/self testing, testing in parts and all together.

- Deployment

Delivering the finished product via various possible methods, such as application or web page, and preparing the customer to receive it, conducting trainings and hardware upgrades if needed.

- Maintenance

Continuing work to address any issues that come up in the use of the software such as bugs or optimization.

- Wrap-up

Summarize and reflect on project happenings, good and bad points, and looking ahead to future projects.

2.4

In version history, for any given version, new additions are highlighted in a color(blue for example), and new removals/subtractions are marked with strikethrough and are faded in color to indicate they have been removed.

In comparison, GitHub versions typically go line-by-line instead, and essentially show both versions a given line side-by-side, highlighting the removed one in red and the new one in green.

2.5

JGBE means “Just Barely Good Enough”, and it is a principle for code documentation. The idea is to not waste too much time going into detail about your implementation while you’re developing, because it is a constantly changing thing.

4.2

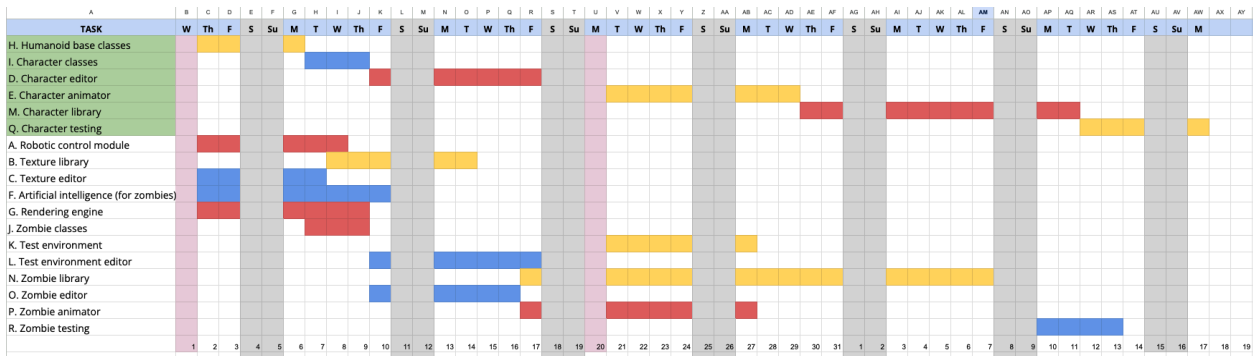
- A. 5 days
- B. 9 days (5 + C’s 4)

- C. 4 days
- D. 12 days (6 + I's 6)
- E. 19 days (7 + D's 12)
- F. 7 days
- G. 6 days
- H. 3 days
- I. 6 days (3 + H's 3)
- J. 6 days (3 + H's 3)
- K. 17 days (5 + L's 12)
- L. 12 days (6 + G's 6)
- M. 28 days (9 + E's 19)
- N. 26 days (15 + O's 11)
- O. 11 days (5 + G's 6)
- P. 17 days (6 + O's 11)
- Q. 32 days (4 + M's 28)
- R. 30 days (4 + N's 26)

The critical path is 32 days long, it's made up of:

- H, I, D, E, M, Q

4.4



4.6

You need to be able to handle unpredictable problems by blocking out more time than you think you need in your estimate, using some rule of thumb like 5% longer, so that when they do come up, you can still stay on schedule and best-case-scenario, there are no delays and you finish earlier than expected!

5.1

Good requirements are clear, unambiguous, consistent with other requirements, prioritized, and verifiable.

5.3

Business, User, Functional, Nonfunctional, Implementation

- a. Allow users to monitor uploads/downloads while away from the office.

Functional, User

- b. Let the user specify website log-in parameters such as an Internet address, a port, a username, and a password.

Functional, User

- c. Let the user specify upload/download parameters such a number of retries if there's a problem.

Functional, User

- d. Let the user select an Internet location, a local file, and a time to perform the upload/download.

Functional, User

- e. Let the user schedule uploads/downloads at any time.

Functional, User

- f. Allow uploads/downloads to run at any time.

Nonfunctional

- g. Make uploads/downloads transfer at least 8 Mbps.

Nonfunctional

- h. Run uploads/downloads sequentially. Two cannot run at the same time.

Functional

- i. If an upload/download is scheduled for a time when another is in progress, it waits until the other one finishes.

Functional

j. Perform schedule uploads/downloads.

Business, Functional

k. Keep a log of all attempted uploads/downloads and whether they succeeded.

Functional, User

l. Let the user empty the log.

Functional, User

m. Display reports of upload/download attempts.

Functional, User

n. Let the user view the log reports on a remote device such as a phone.

Functional, User

o. Send an e-mail to an administrator if an upload/download fails more than its maximum retry number of times.

Functional

p. Send a text message to an administrator if an upload/download fails more than its maximum retry number of times.

Functional

There don't seem to be any business requirements in this list, other than high-level descriptions of the app's functionality. It could be that because this is a productivity tool, its function is pretty straightforward and isn't really in need of a business proposition for people to understand its value.

5.9

My new version of the Hangman game...

MUST

- Allow the user to pick any word they want
- Require the user to sign in to start playing
- Allow the user to connect online to other players to play together

SHOULD

- Allow the user to choose the kind of hangman avatar they would like to use
- Allow the user to set their username and profile to find their friends
- Feature a complete graphics modernization overhaul

COULD

- Include a ranked rating system to compare the best players
- Allow multi-language support and language-based multiplayer lobbies

WON'T

- Introduce voice chat to enable players to trash talk during particularly intense games of hangman
- Auto-translate voice chats into a language of choice for easier trash-talking